

TD Intelligence Artificielle n° 3

Programmation par contraintes

Exercice 1 Rendu de monnaie

On s'intéresse à un distributeur automatique de boissons. L'utilisateur insère des pièces de monnaie pour un total de T centimes d'Euros, puis il sélectionne une boisson, dont le prix est de P centimes d'Euros (T et P étant des multiples de 10). Il s'agit alors de calculer la monnaie à rendre, sachant que le distributeur a en réserve R_{E2} pièces de 2 €, R_{E1} pièces de 1 €, R_{C50} pièces de 50 centimes, R_{C20} pièces de 20 centimes et R_{C10} pièces de 10 centimes.

1. Modélisez la situation sous la forme d'un problème de satisfaction de contraintes (CSP).
2. Comment pourrait-on exprimer le fait que l'on souhaite que le distributeur rende le moins de pièces possible ?
3. Quel intérêt aurait-on à bien choisir l'ordre de sélection des variables et l'ordre de sélection des valeurs ?

Exercice 2 Coloration de graphe

On souhaite colorer les zones australiennes de la figure 1 en rouge, vert et bleu (deux régions adjacentes doivent être de couleurs différentes).



FIGURE 1 – Division de l'Australie en sept zones.

1. Modélisez la situation sous la forme d'un problème de satisfaction de contraintes (CSP).
2. Dessinez le graphe des contraintes (binaires).
3. Appliquez l'algorithme AC-3 jusqu'au traitement du premier arc de l'agenda ($\mathcal{Q}$). Que remarquez-vous ?
4. Appliquez l'algorithme de retour sur trace simple (*backtracking*) jusqu'à obtenir une solution.
5. Appliquez l'algorithme de retour sur trace avec anticipation, version *forward checking*, jusqu'à obtenir une solution.

Exercice 3 Quatre dames

Le problème consiste à placer quatre dames sur un échiquier de taille 4×4 (figure 2), de sorte qu'elles ne puissent se menacer. Dans le jeu d'échecs, une dame peut se déplacer en ligne droite en suivant les colonnes, les rangées (lignes), ou les diagonales du plateau.

Dans cet exercice, on considère la modélisation sous forme de problème de satisfaction de contraintes (CSP) suivante :

- Variables :
 $\mathcal{X} = \{X_1, X_2, X_3, X_4\}$
 X_i : colonne de la i -ième dame, qui se situe sur la i -ième rangée.
- Domaines :
 $\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3, \mathcal{D}_4\}$
 $\mathcal{D}_i = \{a, b, c, d\}$
- Contraintes :
 $\mathcal{C} = \{C_{ij} | 1 \leq i < j \leq 4\}$
 $C_{ij} : X_i \neq X_j \text{ et } |X_i - X_j| \neq |i - j|$
c.-à-d. « la i -ième dame et la j -ième dame ne peuvent se trouver sur la même colonne ou sur la même diagonale »,
c.-à-d. « la i -ième dame et la j -ième dame ne se menacent pas ».

Notez que la définition des variables intègre la contrainte « deux dames ne peuvent se trouver sur la même rangée », et permet d'avoir des domaines dont le cardinal est 4 et non 16.

Lors de l'application d'un algorithme de résolution, vous sélectionnerez les variables selon l'ordre X_1, X_2, X_3, X_4 , et les valeurs selon l'ordre a, b, c, d. Vous garderez la trace des affectations et des domaines en redessinant le plateau à chaque étape.

	a	b	c	d	
1					1
2					2
3					3
4					4
	a	b	c	d	

FIGURE 2 – Plateau associé au problème des quatre dames.

1. Appliquez l'algorithme de retour sur trace avec anticipation, version *forward checking*, jusqu'à remettre en cause l'affectation de la première dame.
2. Appliquez l'algorithme de retour sur trace avec anticipation, version *partial lookahead*, jusqu'à remettre en cause l'affectation de la première dame.
3. Appliquez l'algorithme de retour sur trace avec anticipation, version *full lookahead*, jusqu'à remettre en cause l'affectation de la première dame.

Annexe

Algorithme 1 Retour sur trace

Entrée :

- $(\mathcal{X}, \mathcal{D}, \mathcal{C})$: un CSP sur des domaines finis
- $\mathcal{A}$: une affectation partielle pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$ (pour le 1^{er} appel, $\mathcal{A} = \emptyset$)

Sortie :

- une solution pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$, construite par extension de $\mathcal{A}$
ou
- ECHEC, si $\mathcal{A}$ ne peut pas être étendue en une solution pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$

fonction retourSurTrace($\mathcal{A}, \mathcal{X}, \mathcal{D}, \mathcal{C}$)

/ on teste les contraintes qui lient des variables déjà affectées dans $\mathcal{A}$, et l'on s'arrête si l'une d'elles n'est pas satisfaite */*

si $\mathcal{A}$ n'est pas cohérente **alors retourner** ECHEC

fin si

si toutes les variables de $\mathcal{X}$ sont affectées dans $\mathcal{A}$ **alors**

/ $\mathcal{A}$ est une affectation complète cohérente, c.-à-d. une solution */*

retourner $\mathcal{A}$

sinon

/ $\mathcal{A}$ est une affectation partielle cohérente */*

choisir la 1^{ère} variable $X_i \in \mathcal{X}$ qui n'est pas encore affectée dans $\mathcal{A}$

pour toute valeur $v \in \mathcal{D}_i$ **faire**

$\mathcal{A}' \leftarrow \mathcal{A} \cup \{(X_i, v)\}$

$resultat \leftarrow$ retourSurTrace($\mathcal{A}', \mathcal{X}, \mathcal{D}, \mathcal{C}$)

si $resultat \neq$ ECHEC **alors retourner** $resultat$

fin si

fin pour

/ toutes les branches ont échoué : aucune solution */*

retourner ECHEC

fin si

fin fonction

Algorithme 2 AC-3

Entrée : $(\mathcal{X}, \mathcal{D}, \mathcal{C})$: un CSP sur des domaines finis, avec des contraintes unaires ou binaires

Sortie : domaines modifiés en place pour la cohérence des nœuds et des arcs

fonction ac3($\mathcal{X}, \mathcal{D}, \mathcal{C}$)

/ filtrage des domaines par rapport à la cohérence des nœuds (contraintes unaires) */*

pour tout domaine $\mathcal{D}_i \in \mathcal{D}$ **faire** $\mathcal{D}_i \leftarrow \{v \in \mathcal{D}_i \mid C_i(v) \text{ satisfaite}\}$

fin pour

/ filtrage des domaines par rapport à la cohérence des arcs (contraintes binaires) */*

$\mathcal{Q} \leftarrow \{(i, j) \mid C_{ij} \in \mathcal{C} \text{ ou } C_{ji} \in \mathcal{C}\}$

tant que $\mathcal{Q} \neq \emptyset$ **faire**

sélectionner et supprimer un arc (i, j) de $\mathcal{Q}$

/ la suppression de valeurs dans $\mathcal{D}_i$ peut remettre en cause la cohérence des arcs (k, i) */*

si revise($i, j, \mathcal{X}, \mathcal{D}, \mathcal{C}$) **alors** $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(k, i) \mid (C_{ki} \in \mathcal{C} \text{ ou } C_{ik} \in \mathcal{C}) \text{ et } k \neq j\}$

fin si

fin tant que

fin fonction

Algorithme 3 Fonction de révision

Entrée :

- $(\mathcal{X}, \mathcal{D}, \mathcal{C})$: un CSP sur des domaines finis, avec des contraintes unaires ou binaires
- (i, j) : un arc du graphe des contraintes

Sortie : domaine $\mathcal{D}_i$ modifié en place pour la cohérence de l'arc (i, j) **fonction** revise($i, j, \mathcal{X}, \mathcal{D}, \mathcal{C}$) $revision \leftarrow \text{FAUX}$ **pour toute** valeur $v \in \mathcal{D}_i$ **faire****si** il n'y a pas de valeur $w \in \mathcal{D}_j$ telle que $C_{ij}(v, w)$ ou $C_{ji}(w, v)$ soit satisfaite **alors**
supprimer v de $\mathcal{D}_i$ $revision \leftarrow \text{VRAI}$ **fin si****fin pour****retourner** $revision$ **fin fonction**

Algorithme 4 Forward checking

Entrée :

- $(\mathcal{X}, \mathcal{D}, \mathcal{C})$: un CSP sur des domaines finis, avec des contraintes binaires
- $\mathcal{A}$: une affectation partielle pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$, telle que pour toute $X_i \in \mathcal{X}$ non affectée dans $\mathcal{A}$, pour toute $v \in \mathcal{D}_i$, $\mathcal{A} \cup \{(X_i, v)\}$ est cohérente (pour le 1^{er} appel, $\mathcal{A} = \emptyset$)

Sortie :

- une solution pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$, construite par extension de $\mathcal{A}$
ou
- ECHEC, si $\mathcal{A}$ ne peut pas être étendue en une solution pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$

fonction forwardChecking($\mathcal{A}, \mathcal{X}, \mathcal{D}, \mathcal{C}$)**si** toutes les variables de $\mathcal{X}$ sont affectées dans $\mathcal{A}$ **alors**/* $\mathcal{A}$ est une affectation complète cohérente, c.-à-d. une solution */**retourner** $\mathcal{A}$ **sinon**/* $\mathcal{A}$ est une affectation partielle cohérente */choisir la 1^{ère} variable $X_i \in \mathcal{X}$ qui n'est pas encore affectée dans $\mathcal{A}$ **pour toute** valeur $v \in \mathcal{D}_i$ **faire** $\mathcal{A}' \leftarrow \mathcal{A} \cup \{(X_i, v)\}$ /* filtrage des domaines par rapport à $\mathcal{A}'$ */ $\mathcal{D}' \leftarrow \text{copie}(\mathcal{D})$ **pour toute** variable $X_j \in \mathcal{X}$ qui n'est pas encore affectée dans $\mathcal{A}'$ **faire**/* par hypothèse, $\mathcal{A} \cup \{(X_i, v)\}$ et $\mathcal{A} \cup \{(X_j, w)\}$ sont cohérentes, d'où :« $\mathcal{A}' \cup \{(X_j, w)\}$ est cohérente ssi il n'existe pas $C_{ij} \in \mathcal{C}$ (resp. $C_{ji} \in \mathcal{C}$) t.q. $C_{ij}(v, w)$
(resp. $C_{ij}(w, v)$) soit non satisfaite »la cohérence des $\mathcal{A}' \cup \{(X_j, w)\}$ garantit les hypothèses pour les appels suivants */ $\mathcal{D}'_j \leftarrow \{w \in \mathcal{D}_j \mid \mathcal{A}' \cup \{(X_j, w)\} \text{ est cohérente}\}$ **si** $\mathcal{D}'_j = \emptyset$ **alors retourner** ECHEC**fin si****fin pour** $resultat \leftarrow \text{forwardChecking}(\mathcal{A}', \mathcal{X}, \mathcal{D}', \mathcal{C})$ **si** $resultat \neq \text{ECHEC}$ **alors retourner** $resultat$ **fin si****fin pour**

/* toutes les branches ont échoué : aucune solution */

retourner ECHEC**fin si****fin fonction**

Algorithme 5 Partial lookahead

Entrée :

- $(\mathcal{X}, \mathcal{D}, \mathcal{C})$: un CSP sur des domaines finis, avec des contraintes binaires
- $\mathcal{A}$: une affectation partielle pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$, telle que pour toute $X_i \in \mathcal{X}$ non affectée dans $\mathcal{A}$, pour toute $v \in \mathcal{D}_i$, $\mathcal{A} \cup \{(X_i, v)\}$ est cohérente (pour le 1^{er} appel, $\mathcal{A} = \emptyset$)

Sortie :

- une solution pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$, construite par extension de $\mathcal{A}$
ou
- ECHEC, si $\mathcal{A}$ ne peut pas être étendue en une solution pour $(\mathcal{X}, \mathcal{D}, \mathcal{C})$

fonction partialLookahead($\mathcal{A}, \mathcal{X}, \mathcal{D}, \mathcal{C}$)**si** toutes les variables de $\mathcal{X}$ sont affectées dans $\mathcal{A}$ **alors**/* $\mathcal{A}$ est une affectation complète cohérente, c.-à-d. une solution */**retourner** $\mathcal{A}$ **sinon**/* $\mathcal{A}$ est une affectation partielle cohérente */choisir la 1^{ère} variable $X_i \in \mathcal{X}$ qui n'est pas encore affectée dans $\mathcal{A}$ **pour toute** valeur $v \in \mathcal{D}_i$ **faire** $\mathcal{A}' \leftarrow \mathcal{A} \cup \{(X_i, v)\}$ /* filtrage des domaines par rapport à $\mathcal{A}'$ (comme pour forwardChecking) */ $\mathcal{D}' \leftarrow \text{copie}(\mathcal{D})$ **pour toute** variable $X_j \in \mathcal{X}$ qui n'est pas encore affectée dans $\mathcal{A}'$ **faire** $\mathcal{D}'_j \leftarrow \{w \in \mathcal{D}_j \mid \mathcal{A}' \cup \{(X_j, w)\} \text{ est cohérente}\}$ **si** $\mathcal{D}'_j = \emptyset$ **alors retourner** ECHEC**fin si****fin pour**

/* filtrage des domaines par rapport à la cohérence des futurs arcs */

pour tout couple $(X_k, X_j) \in \mathcal{X}^2$, $k < j$, de variables non encore affectées dans $\mathcal{A}'$ **faire**revise($k, j, \mathcal{X}, \mathcal{D}', \mathcal{C}$)**si** $\mathcal{D}'_k = \emptyset$ **alors retourner** ECHEC**fin si****fin pour** $resultat \leftarrow \text{partialLookahead}(\mathcal{A}', \mathcal{X}, \mathcal{D}', \mathcal{C})$ **si** $resultat \neq \text{ECHEC}$ **alors retourner** $resultat$ **fin si****fin pour**

/* toutes les branches ont échoué : aucune solution */

retourner ECHEC**fin si****fin fonction**

Remarque : *Full lookahead* est identique à *partial lookahead*, hormis la considération sur l'ordre d'affectation des variables, au moment du filtrage des domaines par rapport à la cohérence des futurs arcs ($k \neq j$ au lieu de $k < j$).